

MIPS 五级流水线可视化模拟器设计与性能分析 II

计算机 2302 班 费诗涵 2233221093

一、实验内容与研究目标

本次实验围绕 MIPS 五级流水线的结构化模拟与性能分析展开。与第一次上机实验中直接使用 MIPSsim 观察流水线状态不同，本实验进一步要求设计并实现一个自制流水线模拟器。实验目标并不是单纯让若干条汇编指令顺序执行结束，而是通过自制模拟器明确展示指令如何在 IF、ID、EX、MEM、WB 五个阶段中推进，并观察数据冲突、控制冲突和 Forwarding 机制对流水线性能的影响。

本实验采用“双模拟器”方案。模拟器 A 为本人基于 Python Flask、HTML、CSS 和 JavaScript 实现的 Web 可视化五级流水线模拟器；模拟器 B 继续采用第一次上机实验中使用过的 MIPSsim，用于提供成熟图形化模拟器的对照结果。两者的分工如表 1 所示。

表 1 实验平台与核心观察对象

平台	实现形式	核心观察对象
模拟器 A	Python Flask + Web 前端自制模拟器	五级流水线状态、RAW 检测、Forwarding、Stall、Flush、CPI
模拟器 B	MIPSsim 图形化模拟器	运行同一程序观察结果以验证模拟器 A 是否正确
测试程序	no_hazard.asm、raw_hazard.asm、branch.asm	无明显冲突、RAW 数据冲突、控制冲突

本实验的分析方法分为三个层次。第一层是结构层，即将处理器执行过程抽象为 IF、ID、EX、MEM、WB 五个流水段。第二层是机制层，即说明 RAW 冲突、load-use hazard、分支跳转和 Forwarding 如何改变流水线推进。第三层是性能层，即通过 Cycle Count、Instruction Count、Stall Count、Flush Count 和 CPI 判断不同程序结构对整体执行效率的影响。

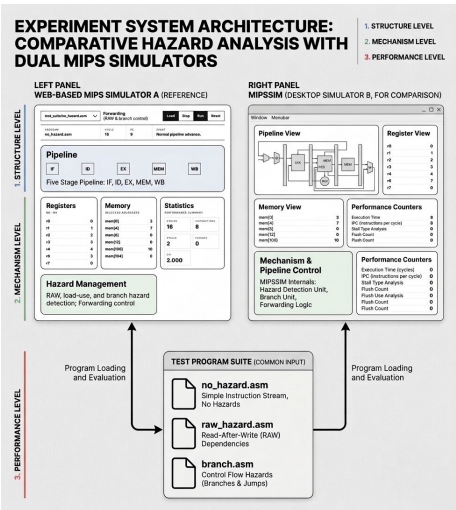


图 1 实验总体方案图：模拟器 A、模拟器 B 与三组测试程序

二、实验环境配置与实践操作

2.1 实验环境组成

本实验在 Windows 11 环境下完成。本实验采用轻量级 Python Flask 后端与原生前端页面完成。

表 2 实验环境配置

项目	配置内容
操作系统	Windows 11
后端语言	Python 3.8.6
Web 框架	Flask
前端技术	HTML、CSS、JavaScript、JSON
开发工具	Visual Studio Code
浏览器	Microsoft Edge 或 Google Chrome
模拟器 B	MIPSSim
主要观察窗口	Web Pipeline View、Register View、Memory View、Statistics View

2.2 项目目录创建

为了避免中文路径、空格路径和同步目录带来的环境问题，本实验将工程放置在英文路径下，采用导入文件的方式运行。目录结构如下：

```
LAB2/
|-- app.py
|-- simulator.py
|-- templates/
|   |-- index.html
|-- static/
|   |-- style.css
|   |-- main.js
```

2.3 虚拟环境与 Flask 安装

为了保证实验环境独立，本实验使用 Python 虚拟环境安装 Flask：

```
python -m venv .venv
.venv\Scripts\activate.bat
pip install flask
python -c "import flask; print(flask.__version__)"
```

若在 PowerShell 中执行激活命令时出现脚本禁止运行问题，应切换到 VS Code 的 Command Prompt 终端，再执行：

```
.venv\Scripts\activate.bat
```

该处理方式比修改全局 PowerShell 执行策略更稳妥，因为它不会影响系统安全策略，也足以满足本实验运行需要。

2.4 Flask 启动方式

本实验网页不能通过双击 index.html 直接运行，而必须由 Flask 启动。原因在于前端页面需要通过 JSON 与 Python 后端交换流水线状态。启动命令如下：

```
python app.py
```

浏览器访问：

```
http://127.0.0.1:5000
```

```
● (.venv) PS E:\university\Grade 3 study\Comupter Architecture\lab2> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 925-064-742
127.0.0.1 - - [30/May/2026 17:12:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:12:26] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:12:27] "GET /static/main.js HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:12:27] "POST /api/load HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:12:27] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [30/May/2026 17:12:30] "POST /api/load HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:12:33] "POST /api/step HTTP/1.1" 200 -
```

图 2 Flask 后端启动成功并监听 127.0.0.1:5000

三、模拟器 A 的总体设计思想与特色

3.1 系统分层结构

模拟器 A 采用前后端分离的轻量级结构。Python 后端负责指令解析、流水线推进、冲突检测、Forwarding 判定和性能统计；Web 前端负责将每个周期的状态以图形化方式展示。两者之间通过 JSON 传递状态。

表 3 模拟器 A 的模块划分

文件	功能	实验意义
simulator.py	五级流水线模拟核心	决定指令如何推进、冲突如何检测
app.py	Flask 路由与 API	连接 Python 模拟器与 Web 前端
index.html	页面结构	展示流水线、寄存器、内存和统计区
style.css	视觉样式	形成图形化实验界面
main.js	前端交互逻辑	实现 Load、Step、Run、Reset 和 Forwarding 切换
programs/*.asm	测试程序	提供无冲突、RAW 冲突和分支冲突场景

综上，即 Python 承担模拟器的计算逻辑，便于与课程中的流水线原理对应；Web 页面只承担可视化展示，便于截图、演示和实验报告表达。

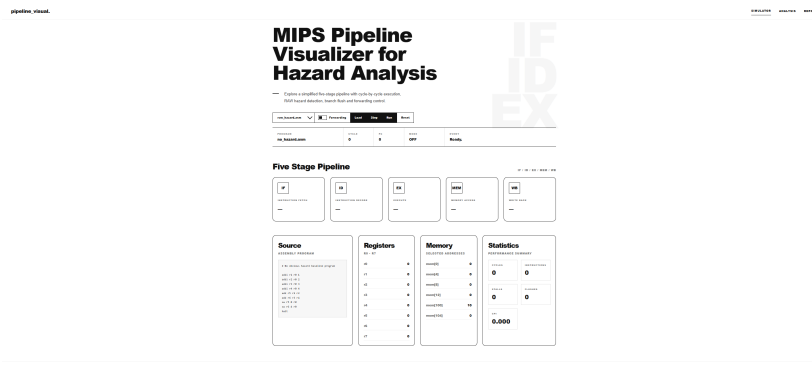


图 3 自制 Web 五级流水线模拟器主界面

3.2 Flask 后端 API 设计

app.py 的作用是提供 Web 页面入口和若干 API。在页面上点击 Load、Step、Run 或 Reset 时，main.js 会向 Flask 后端发送请求；Flask 调用 PipelineSimulator 并将当前状态以 JSON 返回前端。关键代码如下：

```
@app.route("/api/step", methods=["POST"])
def step_program():
    global simulator

    if simulator is None:
        simulator = create_simulator(current_program, current_forwarding)

    simulator.step()
    return jsonify(simulator.get_state())
```

前端每请求一次 /api/step，后端就推进一个时钟周期，并返回新的 IF、ID、EX、MEM、WB 状态。

3.3 JSON 状态传递格式

后端返回的状态包含程序名、周期数、PC、Forwarding 状态、流水线五段状态、寄存器、内存和性能统计。其抽象结构如下：

```
return {
    "program": ...,
    "forwarding": ...,
    "cycle": ...,
    "pc": ...,
    "pipeline": {
        "IF": {"text": ..., "type": ...},
        "ID": {"text": ..., "type": ...},
        "EX": {"text": ..., "type": ...},
        "MEM": {"text": ..., "type": ...},
        "WB": {"text": ..., "type": ...},
    },
}
```

```

    "registers": ...,
    "memory": ...,
    "stats": {
        "cycles": ...,
        "instructions": ...,
        "stalls": ...,
        "flushes": ...,
        "cpi": ...
    }
}

```

可验证本实验中的网页并非静态展示，而是一个可交互的状态观察器。其功能类似 MIPSsim 的 Pipeline View、Register View 和 Statistics View，但内部逻辑由本人实现。

四、流水线核心机制实现

4.1 指令抽象与解析

模拟器 A 采用简化类 MIPS 语法，支持 add、addi、lw、sw、beqz 和 halt。该设计满足实验要求中的 load、store、add、beqz 基本指令集合，同时加入 addi 和 halt 以便构造测试程序和统一终止条件。

指令结构定义如下：

```

@dataclass
class Instruction:
    op: str
    raw: str
    pc: int
    rd: Optional[str] = None
    rs: Optional[str] = None
    rt: Optional[str] = None
    imm: int = 0
    label: Optional[str] = None

```

该结构将一条汇编指令拆分为操作码、源寄存器、目的寄存器、立即数和标签。与直接保存字符串相比，这种结构更适合后续进行寄存器依赖分析。例如，RAW 冲突检测需要知道某条指令读取哪些寄存器、前序指令写入哪个寄存器。

4.2 五级流水线状态表示

模拟器用五个变量表示五个流水段：

```

self.IF = None
self.ID = None
self.EX = None
self.MEM = None
self.WB = None

```

每个流水段并不只保存指令本身，还保存该指令在当前阶段产生的中间结果：

```
def make_stage(self, instr):
    if instr is None:
        return None
    return {
        "instr": instr,
        "alu_result": None,
        "mem_result": None,
        "store_address": None,
        "store_value": None,
        "taken": False,
        "target_pc": None,
    }
```

IF/ID、ID/EX、EX/MEM、MEM/WB 并不只是传递指令文本，还需要传递操作数、ALU 结果、访存结果和控制信号。

4.3 时钟周期推进机制

每执行一次 step()，模拟器推进一个时钟周期。其基本顺序为：先处理旧 WB 阶段写回，再处理旧 MEM 阶段访存，再处理旧 EX 阶段执行，随后根据是否出现分支或数据冲突决定下一周期各段状态。

核心结构如下：

```
old_IF = self.IF
old_ID = self.ID
old_EX = self.EX
old_MEM = self.MEM
old_WB = self.WB

self.write_back_stage(old_WB)
self.memory_stage(old_MEM)
self.execute_stage(old_EX)
```

该结构避免了在同一周期内直接覆盖当前流水段：若直接从 IF 到 WB 顺序更新变量，可能导致后一阶段读取到已经被本周期修改过的状态，从而破坏“同一周期内各阶段并行工作”的语义。

4.4 RAW 冲突检测机制

数据冲突检测的核心是判断 ID 阶段指令将要读取的寄存器，是否正好是 EX 或 MEM 阶段前序指令将要写入的寄存器。代码中通过 get_read_registers() 和 get_dest_register() 完成这一判断：

```
def reads_stage_dest(self, instr, stage):
    if instr is None or stage is None:
        return False

    read_regs = self.get_read_registers(instr)
```

```

read_regs = [reg for reg in read_regs if reg != "r0"]

dest = self.get_dest_register(stage)

if dest is None or dest == "r0":
    return False

return dest in read_regs

```

r0 被排除在冲突检测之外，因为 MIPS 中 r0 恒为 0，不具有真实写入意义。该处理使模拟器更符合 MIPS 寄存器语义，也避免了伪冲突导致的额外 Stall。

4.5 Forwarding 机制设计

关闭 Forwarding 时，只要 ID 阶段读取 EX 或 MEM 阶段将要写入的寄存器，就认为需要停顿：

```

if not self.forwarding:
    if self.reads_stage_dest(id_instr, self.EX):
        return True
    if self.reads_stage_dest(id_instr, self.MEM):
        return True
    return False

```

开启 Forwarding 后，ALU 指令之间的大部分 RAW 相关可以通过旁路解决；但 lw 后紧跟使用结果的 load-use hazard 仍然可能需要插入一个 bubble：

```

if self.forwarding:
    if self.EX is not None:
        ex_instr = self.EX["instr"]
        if ex_instr.op == "lw" and self.reads_stage_dest(id_instr, self.EX):
            return True

```

该实现体现了 Forwarding 不是取消数据依赖，而是缩短结果从前序指令传递到后续指令的路径。对于 load 指令，其数据要到 MEM 阶段才真正可用，因此即使开启 Forwarding，紧邻的后续指令仍可能等待。

4.6 分支跳转与 Flush 机制

beqz 指令在 EX 阶段判断源寄存器是否为 0。如果条件成立，则设置 taken 和 target_pc：

```

elif instr.op == "beqz":
    value = self.get_register_value(instr.rs)
    if value == 0:
        stage["taken"] = True
        stage["target_pc"] = self.labels[instr.label]

```

当 EX 阶段发现分支成立时，模拟器更新 PC，并清空 IF 与 ID 阶段中已经取到的错误路径指

令:

```
if branch_taken:
    self.flush_count += 1
    self.pc = branch_target
    self.finished_fetch = False

    self.WB = old_MEM
    self.MEM = old_EX
    self.EX = None
    self.ID = None
    self.IF = None

    self.last_event = "Branch taken. IF and ID are flushed."
    return
```

上述代码对应控制冲突的基本处理方式：当处理器尚未确定分支方向时，前端可能已经取到了顺序路径上的指令；一旦发现分支成立，错误路径上的指令必须被清除。该过程在本模拟器中体现为 Flush Count 增加。

五、Web 可视化界面设计

5.1 五级流水线卡片显示

前端页面用五个 stage-card 展示 IF、ID、EX、MEM、WB。每个卡片显示阶段名、阶段功能和当前指令：

```
<div class="stage-card" id="stage-IF">
  <span class="stage-name">IF</span>
  <p class="stage-title">Instruction Fetch</p>
  <strong class="stage-instr" id="IF-text">—</strong>
</div>
```

main.js 收到后端 JSON 后，会将每个阶段的指令文本写入对应卡片：

```
function updatePipelineStage(stageName, stageData) {
    const textElement = document.getElementById(`${stageName}-text`);
    const cardElement = document.getElementById(`stage-${stageName}`);

    textElement.textContent = stageData.text;

    if (stageData.type && stageData.type !== "empty") {
        cardElement.classList.add(`op-${stageData.type}`);
    }
}
```

该代码使不同类型指令能够以不同背景色显示。例如，lw 和 sw 属于访存相关指令，beqz 属于

控制流指令，add 和 addi 属于 ALU 指令。

5.2 交互按钮与单周期推进

页面中提供 Load、Step、Run、Reset 和 Forwarding 开关。Step 按钮每次只推进一个周期：

```
async function stepProgram() {
  const state = await postJSON("/api/step", {});
  updateUI(state);
}
```

这满足实验对“单步执行”的要求。与一次性输出结果相比，单步执行可以观察某条指令从 IF 进入 ID、EX、MEM、WB 的全过程，也可以准确定位 RAW 冲突或分支 Flush 出现在哪一个周期。

六、测试程序设计

6.1 无明显冲突基准程序

no_hazard.asm 用于构造相对平稳的流水线基准场景。程序先初始化寄存器，再执行加法和存储：

```
addi r1 r0 1
addi r2 r0 2
addi r3 r0 3
addi r4 r0 4
add r5 r1 r2
add r6 r3 r4
sw r5 0 r0
sw r6 4 r0
halt
```

该程序中没有条件分支，也没有 lw 后紧跟使用结果的 load-use hazard。

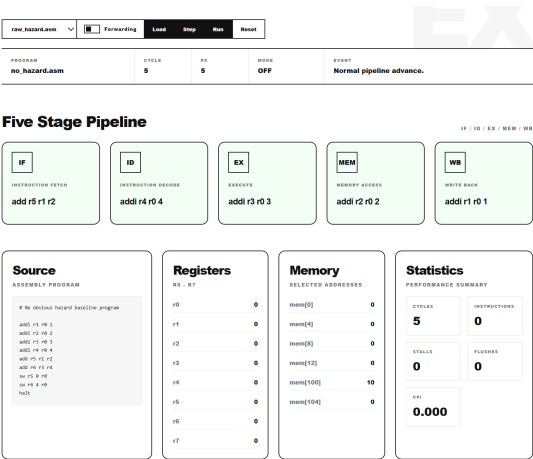


图 4 no_hazard.asm 执行过程中五级流水线稳定推进

6.2 RAW 数据冲突程序

raw_hazard.asm 用于构造典型 load-use RAW 冲突：

```

addi r2 r0 100
lw r1 0 r2
add r3 r1 r1
sw r3 4 r2
halt

```

核心依赖为：

```

lw r1 0 r2
add r3 r1 r1

```

第一条指令从内存地址 $r2 + 0$ 读取数据并写入 $r1$ ，下一条 `add` 立即读取 $r1$ 。因此，从程序顺序看，`add` 必须等待 `lw` 的结果；从流水线时序看，`add` 可能在 `lw` 尚未完成 MEM 访存前进入 EX 阶段，因而产生 RAW 冲突。

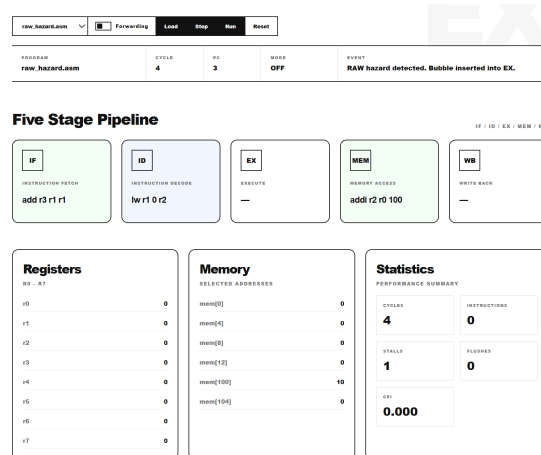


图5 raw_hazard.asm 中 lw 与 add 之间的 RAW 冲突

6.3 分支跳转程序

branch.asm 用于展示 `beqz` 引起的控制冲突：

```

addi r1 r0 2
loop:
addi r1 r1 -1
beqz r1 end
add r2 r2 r1
beqz r0 loop
end:
halt

```

该程序中 $r1$ 初值为 2，每轮循环递减。当 $r1$ 变为 0 时，`beqz r1 end` 成立，程序跳转到 `end`。由于分支方向需要到 EX 阶段才能确定，因此在分支判断完成前，IF 和 ID 阶段可能已经取入顺序路径上的指令。一旦分支成立，这些指令必须被清空，从而形成 Flush。

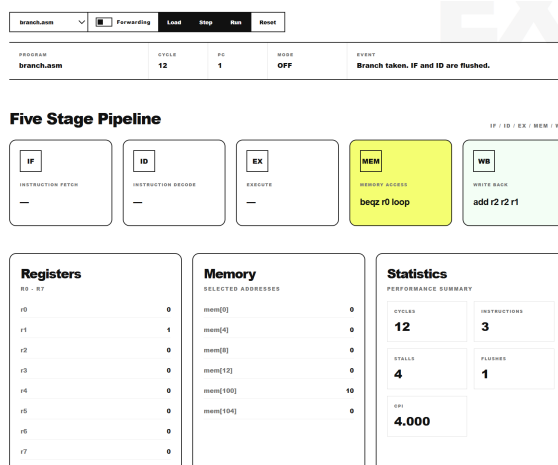


图6 branch.asm 中 beqz 分支成立后 IF 与 ID 阶段被清空

七、模拟器 A 的运行结果与分析

7.1 无明显冲突程序的执行结果

在 no_hazard.asm 中，指令能够较稳定地进入五个流水段。连续单步执行后，可以观察到多条指令同时分布在 IF、ID、EX、MEM 和 WB 中，说明模拟器能够正确体现流水线“阶段重叠”的基本思想。

运行步骤如下：

1. 浏览器打开 <http://127.0.0.1:5000>
2. Program 选择 no_hazard.asm
3. Forwarding 保持 OFF
4. 点击 Load
5. 多次点击 Step，观察 IF/ID/EX/MEM/WB
6. 点击 Run，观察最终 Statistics

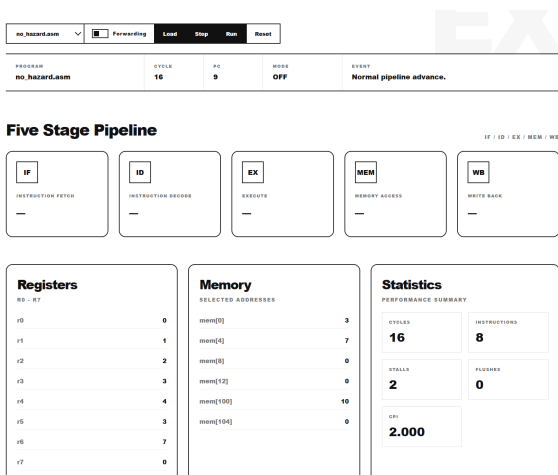


图7 no_hazard.asm 执行结束后的周期数、指令数、Stall 数与 CPI

实验现象表现为：Stall 数较低，Flush 数为 0，CPI 相比冲突程序更接近理想流水线。该结果说

明，在没有紧邻数据依赖和控制流改变的程序中，流水线能够通过阶段重叠提高指令吞吐率。

7.2 RAW 冲突在 Forwarding OFF 下的表现

关闭 Forwarding 后运行 raw_hazard.asm。由于 add r3 r1 r1 需要读取 r1，而 r1 由前一条 lw 产生，模拟器会在检测到 RAW 相关后向 EX 阶段插入 bubble，同时保持 IF 和 ID 不动。

对应代码逻辑为：

```
if hazard:
    self.stall_count += 1

    self.WB = old_MEM
    self.MEM = old_EX
    self.EX = None
    self.ID = old_ID
    self.IF = old_IF

    self.last_event = "RAW hazard detected. Bubble inserted into EX."
    return
```

这段代码直接对应流水线停顿行为：后续指令不再继续前进，而 EX 阶段被插入空泡。实验界面中可以通过 Event 和 Stalls 的变化观察该现象。

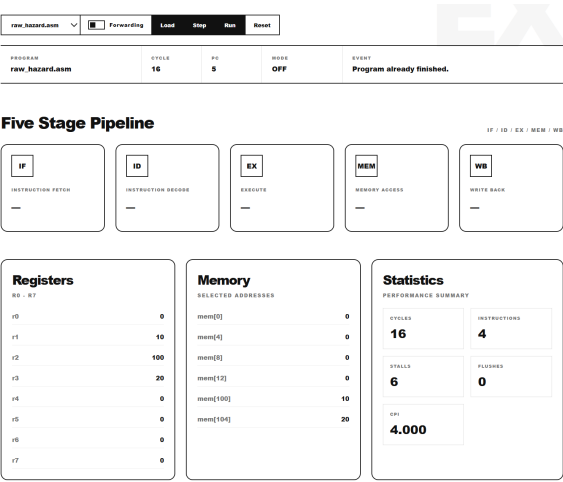


图 8 raw_hazard.asm 在 Forwarding OFF 下的性能统计

7.3 RAW 冲突在 Forwarding ON 下的改善

开启 Forwarding 后，再次运行 raw_hazard.asm。实验结果应表现为 Stall 数下降，但不一定完全为 0。原因在于该程序构造的是 lw 后紧跟 add 的 load-use hazard。load 指令的数据需要在 MEM 阶段后才真正可用，因此即使开启 Forwarding，仍可能至少需要一个周期等待。

从理论上讲，Forwarding 的作用是缩短数据从生产者指令到消费者指令的传递路径，而不是改变指令之间的语义依赖。对于 ALU 运算结果，Forwarding 通常可以明显减少等待；对于 load-use 场景，由于数据产生时间更晚，停顿可能仍然存在。

Performance Comparison

Run the same program twice and compare the behavior with forwarding disabled and enabled.

raw_hazard.asm

Forwarding OFF / ON

Forwarding OFF		Forwarding ON	
Cycles	16	Cycles	11
Instructions	4	Instructions	4
Stalls	6	Stalls	1
Flushes	0	Flushes	0
CPI	4.000	CPI	2.750

Mini Charts



图 9 raw_hazard.asm 在 Forwarding 前后的性能对比

7.4 分支程序中的控制冲突

运行 branch.asm 时, beqz 指令会改变 PC。当分支成立时, 模拟器将 IF 和 ID 阶段清空, 并将 PC 改为目标标签地址。控制冲突的性能损失体现为 Flush Count 增加。

实验步骤如下:

1. Program 选择 branch.asm
2. 点击 Load
3. 点击 Step, 观察 r1 逐步递减
4. 继续单步执行至 beqz r1 end 成立
5. 观察 Event、Flushes 与 IF/ID 清空现象

该结果说明, Forwarding 主要解决数据操作数传递问题, 而分支冲突的本质是下一条 PC 不确定。即使操作数可以通过 Forwarding 提前获得, 处理器仍需等待分支判断结果决定后续取指方向。

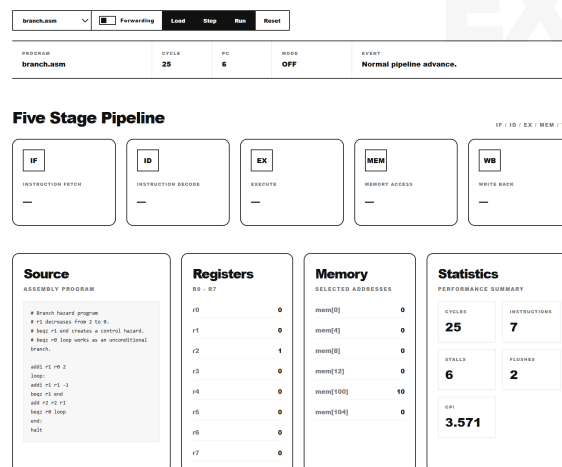


图 10 branch.asm 执行结束后的 Flush Count 与 CPI

7.5 模拟器 A 性能统计汇总

本实验通过 Analysis 页面直观记录每组程序的执行结果。由于不同实现对 halt 是否计入指令数、分支 Flush 时机以及寄存器写回时机的处理可能不同，本文更关注不同场景之间的趋势比较，而不是将每一个周期数绝对化。

表 4 模拟器 A 中不同程序的性能统计汇总

程序	Forwarding	Cycles	Instr.	Stalls	CPI	主要现象
no_hazard.asm	OFF	16	8	2	2.000	阶段重叠稳定
raw_hazard.asm	OFF	16	4	6	4.000	RAW 停顿明显
raw_hazard.asm	ON	11	4	1	2.750	Stall 减少
branch.asm	OFF	25	7	6	3.571	分支 Flush
branch.asm	ON	19	7	0	2.714	分支 Flush

八、模拟器 B: MIPSsim 对照分析

8.1 MIPSsim 作为对照平台的意义

第一次实验中已经使用 MIPSsim 对 data_hz.s、branch.s、pipeline.s 和无冲突基准程序进行了分析。本实验继续将 MIPSsim 作为模拟器 B，作为成熟图形化工具对照验证模拟器 A 结果是否正确。

MIPSsim 的优势在于界面完整，能够直接展示代码窗口、流水线状态窗口、寄存器窗口、内存窗口和统计窗口。相较于 MIPSsim 自制模拟器 A 的优势在于代码逻辑透明，可以清楚解释每一个 Stall 和 Flush 是如何由程序逻辑产生的。



图 11 MIPSsim 模拟器主界面与流水线观察窗口

8.2 MIPSsim 中 RAW 冲突的对照现象

在 MIPSsim 中，data_hz.s 的核心冲突来自 LW 指令后紧跟使用结果的 ADD 或 SW 指令。其原理与本实验 raw_hazard.asm 一致：前一条指令尚未完成写回，后一条指令已经需要读取该寄存器。

```

LW    $r1,0($r2)
ADD   $r1,$r1,$r3
SW    $r1,0($r2)

```

该结构与模拟器 A 中的：

```

lw r1 0 r2
add r3 r1 r1

```

具有相同的分析本质，均属于写后读数据相关。两者区别在于：MIPSSim 支持更完整 MIPS 指令和图形化流水线时序，而模拟器 A 使用简化指令集，以便突出冲突检测和状态可视化。

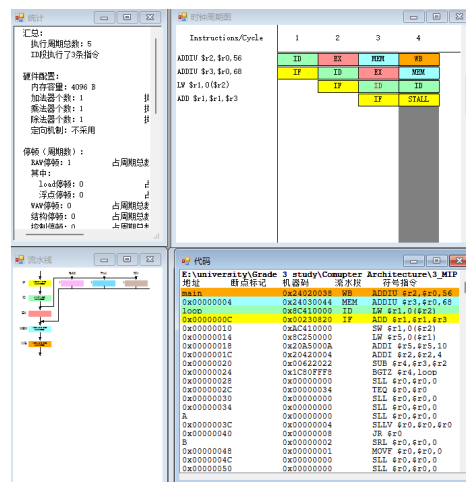


图 12 MIPSsim 中 data_hz.s 的 RAW 数据冲突现象

8.3 模拟器 A 与模拟器 B 的对比

表 5 自制 Web 模拟器 A 与 MIPSsim 模拟器 B 对比

对比项	模拟器 A	模拟器 B
实现方式	Python Flask + HTML/CSS/JS 自制	已有 MIPSsim 图形化模拟器
指令支持	简化类 MIPS 指令	较完整 MIPS 教学指令
流水线显示	Web 卡片显示 IF/ID/EX/MEM/WB	图形化流水线窗口
单步执行	Step 按钮逐周期推进	模拟器内置单步执行
Forwarding	前端开关控制后端逻辑	模拟器设置项控制
性能统计	Cycles、Instructions、Stalls、Flushes、CPI	Statistics View 提供统计
主要优势	逻辑透明，可解释每个冲突处理分支	工具成熟，显示细节完整
主要局限	指令集和硬件模型简化	内部实现不便修改和扩展

九、实验中的典型问题与原理性讨论

9.1 为什么不直接用 JavaScript 实现完整模拟器

答：本实验选择 Python 实现核心逻辑，是因为流水线推进、冲突检测和性能统计更接近算法逻辑，使用 Python 更便于对照课程原理进行解释；JavaScript 主要负责页面交互和状态刷新。若完全使用 JavaScript 实现，虽然可以减少后端，但代码结构会将模拟逻辑与界面逻辑混合在一起，不利于实验报告中的机制分析。

9.2 为什么网页不能直接单击 Go live 打开

答：本实验前端需要调用 /api/step、/api/run 等 Flask 后端接口。若直接双击 index.html，浏览器只能打开静态页面，无法与 Python 模拟器通信，也就无法获得每个周期的流水线状态。因此必须执行 python app.py，并通过 http://127.0.0.1:5000 访问。

9.3 为什么 Forwarding ON 后 Stall 不一定为 0

答：Forwarding 可以将 EX/MEM 或 MEM/WB 阶段的结果提前传递给后续指令，但不能改变数据真正产生的时间。对于 ALU 运算结果，Forwarding 通常能显著减少等待；对于 lw 后紧跟使用结果的 load-use hazard，数据到 MEM 阶段后才可用，因此仍可能需要插入一个 bubble。

9.4 为什么分支冲突不能完全依靠 Forwarding 解决

答：Forwarding 解决的是寄存器操作数传递问题，而分支冲突的核心是下一条 PC 不确定。即使分支使用的寄存器值可以被提前获得，处理器仍需判断分支是否成立；若前端已经取入错误路径指令，就必须 Flush。因此控制冲突和数据冲突是两类不同问题。

9.5 为什么自制模拟器与 MIPSsim 的周期数可能不完全一致

答：不同模拟器对 halt 是否计入指令数、寄存器写回在时钟前半拍还是后半拍、分支延迟槽、Flush 范围和 Forwarding 路径的处理可能不同。因此，模拟器 A 与 MIPSsim 不要求每个周期数完全一致。更重要的是趋势一致：无冲突程序 CPI 较低，RAW 程序关闭 Forwarding 时 Stall 较多，开启 Forwarding 后 Stall 减少，分支程序存在 Flush 开销。

十、进阶设计与拓展分析

10.1 已实现的 Analysis 模块：Forwarding 前后自动对比

在完成五级流水线模拟、单步执行、Forwarding 开关、寄存器与流水段状态显示等基础功能后，本实验进一步实现了 Analysis 模块。该模块并不改变流水线本身的执行逻辑，而是面向实验分析过程，对同一段测试程序分别在 Forwarding OFF 和 Forwarding ON 两种模式下自动运行，并汇总 Cycles、Instructions、Stalls、Flushes 和 CPI 等统计结果。

其核心思想是控制变量：程序文件、初始寄存器状态、初始内存状态和结束条件保持一致，只改变 Forwarding 开关。这样可以避免把程序输入差异误认为性能差异，使 Forwarding 对流水线性能的影响更加直观。

```
@app.route("/api/compare", methods=["POST"])
def compare_program():
    data = request.get_json()
    program_name = data.get("program")

    sim_off = create_simulator(program_name, forwarding=False)
    while not sim_off.is_done():
        sim_off.step()
```



```

sim_on = create_simulator(program_name, forwarding=True)
while not sim_on.is_done():
    sim_on.step()

return jsonify({
    "off": sim_off.get_state()["stats"],
    "on": sim_on.get_state()["stats"]
})

```

在前端页面中，Analysis 模块将两组结果分别显示为 Forwarding OFF 和 Forwarding ON 两个统计卡片，并进一步用简易条形图比较 Cycles、Stalls 和 CPI 的变化。例如对于无明显冲突的基准程序，Forwarding ON 后 Stall 数减少，CPI 也随之下降。对于包含 RAW 数据相关的程序，该模块能够更直接地体现定向转发对数据冲突的缓解作用。

10.2 已实现的 Report 模块：实验报告素材自动生成

除性能对比外，本实验还实现了 Report 模块，用于根据当前程序和运行统计结果自动生成实验记录文字。该模块主要包括两部分功能：一是生成可直接整理进报告的统计摘要与分析说明；二是给出建议截图清单，提示在实验报告中应保留哪些关键证据。

Report 模块生成的内容主要包括程序名称、Forwarding 状态、总周期数、指令数、Stall 数、Flush 数和 CPI，并根据程序类型生成对应的现象解释。例如无明显冲突程序主要用于观察五级流水线稳定重叠执行；RAW hazard 程序主要用于说明前后指令之间的数据依赖；分支程序则用于分析控制相关和 Flush 的产生原因。

10.3 尚未实现的拓展设想

当前系统已经能够完成基本五级流水线模拟、Forwarding 控制、Stall/Flush 统计、性能对比和报告素材生成。后续如果需要更成熟的改进，笔者将参考 MIPSsim 的交互方式，补充更完整的运行控制和状态观察功能。

表 6 后续可拓展但本次尚未实现的功能

拓展方向	说明
断点执行	支持在指定指令或指定 Cycle 处暂停，便于观察某一次 RAW 冲突或分支跳转发生时的流水线状态
横向流水线时序表	自动生成类似 MIPSsim 的 Cycle-Time 表格，用 IF、ID、EX、MEM、WB 标出每条指令在各周期的位置
寄存器与内存快照对比	在程序运行前后保存 Register File 和 Data Memory 状态，用于验证程序语义是否正确
数据前递路径可视化	在页面中用箭头标出 EX/MEM 到 EX、MEM/WB 到 EX 的 Forwarding 路径，使数据旁路关系更加直观
Hazard 日志	自动记录每一次 Stall 或 Flush 的触发周期、相关指令和原因，例如 load-use hazard 或 branch flush
分支预测模拟	增加 always-not-taken、always-taken 或一位预测器，比较不同预测策略下 Flush 次数和 CPI 的变化

十一、实验复现中的常见错误与后辈指导

11.1 没有激活虚拟环境就安装 Flask

若没有看到命令行前缀中的(.venv)，执行 `pip install flask` 可能会把 Flask 安装到系统 Python 中。后续 VS Code 选择 .venv 解释器时仍可能提示 No module named flask。正确顺序应为：

```
python -m venv .venv
.venv\Scripts\activate.bat
pip install flask
```

11.2 直接单击 Go Live 运行网页

这种做法只能打开静态页面，无法调用 Python 后端。正确做法是：

```
python app.py
```

然后在浏览器访问：

```
http://127.0.0.1:5000
```

11.3 修改 simulator.py 后没有重启 Flask

Flask debug 模式通常会自动重载，但当模拟器状态已经保存在后端全局变量中时，修改核心代码后仍建议手动停止并重新执行 `python app.py`。这样可以避免旧 simulator 对象继续存在，导致调试结果与代码不一致。

11.4 误以为 Forwarding 会消除所有冲突

Forwarding 只能减少一部分数据相关造成的等待，不能消除控制冲突，也不能完全消除 load-use hazard。因此，若开启 Forwarding 后仍有 Stall，不应立即判断为程序错误，而应先观察是否存在 lw 后紧跟使用结果的情况。

十二、实验总结与感悟

本实验在第一次实验使用 MIPSsim 观察五级流水线现象的基础上，进一步实现了一个自制 Web 可视化 MIPS 五级流水线模拟器。

从功能实现看，simulator.py 将指令抽象为结构化对象，并用 IF、ID、EX、MEM、WB 五个阶段变量表示流水线状态。step() 函数通过保存旧阶段状态，模拟同一周期内各流水段并行工作的语义。RAW 冲突检测通过比较 ID 阶段读取寄存器和 EX/MEM 阶段写寄存器完成；Forwarding 机制在允许旁路传递的同时保留 load-use hazard 的必要停顿；分支成立时通过清空 IF 和 ID 阶段体现控制冲突中的 Flush。

从实验结果看，no_hazard.asm 展示了五级流水线稳定重叠执行的基本形态；raw_hazard.asm 展示了 lw 后紧跟 add 所造成的 RAW 数据冲突，并说明 Forwarding 能降低但不一定完全消除 Stall；branch.asm 展示了 beqz 改变 PC 后错误路径指令被清空的控制冲突现象。模拟器 B 即 MIPSsim 则作为对照平台，验证了自制模拟器 A 所展示的冲突趋势与成熟教学模拟器中的流水线现象一致。

整体而言，本次实验不仅让我从“写汇编”进一步走向“模拟硬件”，也让我在代码层面真正理解了流水线各阶段协同工作的细节。更重要的是，通过动手实现冲突检测与转发逻辑，我对“为什么流水线需要硬件支持才能高效运行”有了更深的体会。这种从现象到机制、从机制到实现的学习路径，对构建扎实的计算机体系结构思维具有非常重要的意义。